

SEMINÁRIO 01 | TEMA 09

bcrypt

Como guardar senhas do jeito certo: sal, custo e a implementação do algoritmo em Python puro

Isabela Sousa & Paulo Victor Almeida

```
$2b$05$R JatSMavAUXuKMySN8WKfe0eoSNcdW5 . UBnE0m9EnZpnpV64Fe0eq
```

O que é o bcrypt?

Função de hash feita para guardar senhas. Criada em 1999 por Niels Provos e David Mazières, baseada na cifra Blowfish.

Hash de mão única

Da senha se chega ao hash, mas do hash não se volta à senha. No login, o site só compara hashes.

Sal aleatório

16 bytes únicos por senha. A mesma senha gera hashes diferentes e as rainbow tables perdem o efeito.

Custo ajustável

O trabalho é repetido 2^{custo} vezes. Cada +1 dobra o tempo, então ele acompanha computadores mais rápidos.

Anatomia do hash guardado no banco (60 caracteres)

\$2b\$

versão

05

custo

RJatSMavAUXuKMySN8WKfe

sal (22 chars)

0eoSNcdW5.UBnE0m9EnZpnpV64Fe0eq

hash (31 chars)

Os 6 passos do algoritmo

Como o `bcrypt_seminario.py` gera um hash

1 Preparar a senha

Vira bytes UTF-8, ganha um byte `\0` no fim e é cortada em 72 bytes.

```
hash_bruto() [passo 1]
```

2 Estado inicial

Vetor `P` e caixas `S` começam com os dígitos de π (calculados com a fórmula de Machin).

```
Estado() e tabelas_de_pi()
```

3 Misturar sal e senha

`ExpandKey(sal, senha)`: reescreve o estado com 521 cifragens Blowfish.

```
expandir_chave()
```

4 Repetir 2^{custo} vezes

A cada volta: `ExpandKey(senha)` e `ExpandKey(sal)`. Aqui mora a lentidão.

```
laço dentro de hash_bruto()
```

5 Cifrar o texto fixo

'OrpheanBeholderScryDoubt' (3 blocos) é cifrado 64 vezes com o estado final.

```
hash_bruto() [passo 5]
```

6 Montar o hash

`$2b$ + custo + base64(sal) + base64(23 bytes de saída)`.

```
gerar_hash()
```

Fluxo completo da função bcrypt

gerar_hash(senha, custo) -> string de 60 caracteres



Cada ExpandKey = 521 cifragens Blowfish (16 rodadas de Feistel). Com custo 5: 34.057 cifragens no total.

O sal e o custo ficam dentro do hash: é por isso que o login consegue refazer o cálculo.

Exemplo prático: senha 'senhaeu', custo 5

Valores reais do bcrypt_seminario.py (menu, opção 1: gerar hash passo a passo)

- | | | |
|---|------------------------|---|
| 1 | Chave | 'senhaeu' + \0 = 8 bytes |
| | Sal | 4cb72f50e73109667030ed143fe60c86 |
| 2 | Estado = π | P[0] = 243F6A88 |
| 3 | ExpandKey(sal, senha) | P[0] = 74F1E7CD |
| 4 | Após $2^5 = 32$ voltas | P[0] = 03CCDF0A |
| 5 | 64 cifragens do texto | 420A943DE7D8EC05... (24 B) |
| 6 | Hash final | \$2b\$05\$RJatSMavAUXuKMySN8WKfe0eoSNcdW5.UBnE0m9EnZpnpV64Fe0eq |

E no login?

1. Lê o custo (05) e o sal de dentro do hash guardado.
2. Refaz o cálculo com a senha digitada, usando esse mesmo sal.
3. Compara em tempo constante: iguais = acesso liberado.

Total: 34.057 cifragens Blowfish. Uma letra diferente na senha muda o hash por completo.

Por que o bcrypt é seguro?

Ele não impede o palpite: ele torna cada palpite caro

Sal único por senha

Rainbow tables inúteis. O atacante precisa atacar cada usuário separadamente.

Custo que acompanha o hardware

Cada +1 dobra o esforço. Custo 6: ~500 mil tentativas/s. Custo 12: ~8.300/s (cerca de 60x menos). Recomendado: 12 (mín. 10; 14 em sistemas críticos).

Etapas em sequência

Cada cifragem depende do estado alterado pela anterior. Não dá para paralelizar nem pular passos.

Comparação em tempo constante

O tempo de resposta não revela quantos bytes acertou (evita timing attack).

Fonte dos números: L. Lima, Medium (2025); recomendação baseada na OWASP.

Vantagens e limitações

Vantagens

Projetado para senhas, lento de propósito

Sal aleatório já embutido no hash

Custo ajustável com o tempo

Em uso desde 1999, suporte em todas as linguagens

Tudo (versão, custo, sal, hash) em 60 caracteres

Limitações

Limite de 72 bytes: o resto da senha é ignorado

Usa só ~4 KB de memória: mais exposto a GPU e ASIC

Byte nulo (\0) proibido na senha

Senha fraca continua fraca; custo 6 a 8 é inseguro

scrypt e Argon2 exigem memória e resistem melhor a GPU/ASIC

bcrypt x scrypt x Argon2id

Três funções feitas para senhas: o que muda entre elas

	bcrypt	scrypt	Argon2id
Ano	1999	2009	2015
Velocidade	Lento de propósito	Lento + usa memória	Lento + usa memória
Sal embutido	Sim (16 bytes)	Sim	Sim
Custo ajustável	2^{custo}	N, r, p	tempo, memória, threads
Memória exigida	~4 KB (fixa)	Configurável	Configurável
Aceito pela OWASP	Sim (custo 10+)	Sim ($N = 2^{17}$)	Sim (19 MiB, 2 iter.)

Fonte: Psono, A Evolução da Hashing de Senhas (2024), com parâmetros da OWASP. Ano do scrypt: conhecimento geral.

Referências

PROVOS, N.; MAZIÈRES, D. A Future-Adaptable Password Scheme. USENIX, 1999.
usenix.org/legacy/events/usenix99/provos/provos.pdf

AUTH0. Hashing in Action: Understanding bcrypt. auth0.com/blog/ hashing-in-action-understanding-bcrypt

JUMPCLOUD. What is bcrypt? jumpcloud.com/it-index/what-is-bcrypt

LIMA, L. O erro comum que coloca suas senhas em risco: bcrypt com rounds baixos. Medium, 2025.

PSONO. A Evolução da Hashing de Senhas (S. Pfeiffer, 2024). psono.com/pt/blog/evolution-of-password-hashing

Código do trabalho: `bcrypt_seminario.py` (bcrypt em Python puro, sem nenhum import).